Hong Kong Metropolitan University

COMP2090SEF

Data Structures, Algorithms and Problem Solving

Course Project Report

Group 36

Submission date: 4/10/2026

We declare that:

(i) all members of the group have read and checked that all parts of the project (including proposal, code programs, and reports), irrespective of whether they are contributed by individual member or all members as a group, here submitted is original except for source material explicitly acknowledged.

(ii) the project, in parts or as a whole, has not been submitted for more than one purpose without declaration.

(iii) we are aware of the University's policy and regulations on honesty in academic work and understand the possible consequences when breaching such policy and regulations.

(iv) we confirm that we have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and

(v) we are aware that all members of the group should be held responsible and liable to disciplinary actions, irrespective of whether he/she has signed the declaration and whether he/she has contributed, directly or indirectly, to the problematic contents.

We confirm that we have read through and understood the project requirements. We understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
XIA Weihao (PL)	14167751
SITU Ruiyuan	14186925
Shi Zhenghang	14058130

This report is for TASK 2

1. Introduction & Background

In modern finance, the foreign exchange (FX) market is constantly monitored by High-Frequency Trading (HFT) institutions and quantitative funds. They deploy low-latency algorithms to scan for temporary pricing inefficiencies across different decentralized exchanges. These fleeting discrepancies create "closed-loop arbitrage" opportunities (e.g. USD →EUR →GBP → USD), allowing traders to secure risk-free profits while simultaneously enforcing global pricing parity (Foucault et al., 2013). Mathematically, an arbitrage opportunity exists only when the sequential product of exchange rates along a closed loop strictly exceeds one.

To systematically detect these opportunities, the FX market is naturally modeled as a weighted directed graph: currencies act as vertices, one-way exchange channels as directed edges, and exchange rates as edge weights. However, standard path-finding algorithms like Dijkstra's are inadequate for this problem. Dijkstra's algorithm is designed to find the minimum *sum* of positive edge weights and structurally fails when negative weights or cycles are introduced. Arbitrage, conversely, requires maximizing a continuous *product*.

By converting exchange rates into their negative natural logarithms **(*-ln (Rate)*)**, we mathematically transform the search for a profitable loop (product > 1) into a search for a negative weight cycle (sum < 0). This is why the problem demands the Bellman-Ford algorithm. Beyond mere theoretical fit, Bellman-Ford is uniquely equipped to handle negative edge weights and its final relaxation phase deterministically isolates negative cycles (Cormen et al., 2009)—perfectly mirroring the financial logic of arbitrage detection.

Therefore, the objective of this project is to implement an automated Python system using a weighted directed graph and the Bellman-Ford algorithm to efficiently detect closed-loop FX arbitrage routes.

2. Data Structure: Weighted Directed Graph

To model asymmetrical and interconnected markets, this project uses a Weighted Directed Graph, ***G = (V, E)***. Vertices represent tradable fiat currencies, directed edges represent one-way conversion channels, and weights represent transformed exchange rates **(*-ln (Rate)*)**.

The graph is implemented via an Adjacency List because the global currency market is relatively sparse. An Adjacency Matrix would needlessly require $O(V^2)$ space, whereas an Adjacency List optimizes space complexity to ***O(V+E)*** and allows highly efficient edge iteration. The Python ArbitrageGraph class encapsulates this structure.

(Please refer to Appendix A for the detailed implementation methods).

3.Algorithm: Bellman-Ford & Transformation

Standard algorithms cannot process negative weights or maximize products. To utilize pathfinding algorithms, we transform the "multiplication" problem of exchange rates into an "addition" problem using natural logarithms.

By assigning each edge (u, v) a weight of $\varpi(u, v) = -\ln(Rate)$, any closed loop whose sum of weights is strictly less than zero mathematically guarantees a profitable arbitrage path.

(Please refer to Appendix B for the mathematical proof of this transformation).

The Bellman-Ford algorithm natively supports these negative weights and operates in $O(V \cdot E)$ time through three stages:

1. **Initialization:** Distances from the source node are set to 0, and all others to infinity.
2. **Edge Relaxation:** The algorithm iterates through every edge $|V| - 1$ times, updating the shortest path to each node and tracking the route via a predecessor dictionary.
3. **Negative Cycle Detection:** A final relaxation pass is executed. If any distance can still be reduced, a negative weight cycle exists. The system then traces the predecessor dictionary backward to reconstruct the chronological trading route.

Code details are illustrated in Image-b: code of Algothone

4. Example Application & Results

We simulated a triangular arbitrage opportunity using three currencies (GBP, EUR, USD).

4.1 Test Case Setup

The ArbitrageGraph is populated with the following exchange rates and transformed logarithmic weights **(*-ln (Rate)*)**:

- GBP → EUR: 1.20 (Weight: -0.182)
- EUR → USD: 1.10 (Weight: -0.095)
- USD → GBP: 0.90 (Weight: +0.105)

4.2 Algorithm Execution & Path Reconstruction

Running the Bellman-Ford algorithm initializes ***distance [GBP] = 0***. After $|V| - 1$ relaxation rounds, distances update to ***distance [EUR] = -0.182*** and ***distance [USD] = -0.277***.

In the final detection round, traversing the USD → GBP edge yields a new distance of -0.172, mathematically proving a negative weight cycle exists. By tracing the predecessor dictionary ({EUR: GBP, USD: EUR, GBP: USD}) backward, the system reconstructs the exact trading route: **GBP → EUR → USD → GBP**.

4.3 Practical Execution & Profit

Executing this sequence with £1,000 initial capital involves three rapid trades:

1. £1,000 × 1.02 = €1,200
2. €1,200 × 1.02 =$1,320
3. $1,320 × 0.90= £1,188

The product of the original rates (1.20 × 1.10 × 0.90 = 1.188) strictly exceeds 1, yielding a risk-free gross profit of 18.8% (£188) and successfully validating the computational model.

See Appendix C for the full trading route reconstruction and terminal output.

5.Conclusion & Limitations

5.1 Conclusion

This project successfully implemented a Python system using a Weighted Directed Graph and the Bellman-Ford algorithm to identify FX arbitrage. By transforming exchange rates into negative logarithmic weights, the model accurately detects profitable trading routes via negative weight cycles.

5.2 Limitations

The model has practical constraints for High-Frequency Trading (HFT): it assumes a frictionless market (ignoring fees and slippage), relies on static input data, and its single-threaded ***O(V •E)*** complexity is too slow to process massive, modern cryptocurrency networks.

5.3 Future Directions and Self-Reflection

To become a viable HFT tool, the system requires faster queue-based algorithms (e.g., SPFA) or GPU acceleration, real-time WebSocket API integration for dynamic rates, and enhanced code robustness through rigorous unit testing and input validation.

6. AI usage:

Only for parts of data structure and algorithms that is completely beyond our ability coding and debugging

7.References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

Foucault, T., Pagano, M., & Röell, A. (2013). *Market Liquidity: Theory, Evidence, and Policy*. Oxford University Press.

Appendix

Workload

Members	Task 2
XIA Weihao(PL)	Report revision
SITU Ruiyuan	Algorithms&Data structure analyze&video
SHI Zhenghang	Ideas for Algorithms & Code debugging

Appendix A: Core Data Structure Methods

The following table summarizes the core methods implemented in the ArbitrageGraph class.

Method	Complexity	Description
__init__(self)	***O(1)***	Initializes the adjacency list (self.graph), a set to track unique vertices, and a dictionary for original rates.
add_exchange_rate	***O(1)***	Adds a directed edge between base and target currency. Applies the **(*ln(Rate)*)** transformation to store the edge weight.
detect_arbitrage	***O(V •E)***	Executes the Bellman-Ford algorithm. Relaxes edges $
_reconstruct_cycle	***O(V)***	Private helper method. Traces backward through the predecessor mapping to determine the chronological trading route and computes the actual profit multiplier.

Appendix B: Mathematical Proof of Transformation

To process an arbitrage search using a pathfinding algorithm, we must transform a maximization problem into a minimization problem using logarithmic properties.

Step	Mathematical Formula	Logical Explanation
1. Identify Objective	$R_1 \times R_2 \times \cdots \times R_n > 1$	Arbitrage exists if the product of sequential exchange rates is strictly greater than 1.
2. Log Transformation	$ln(R_1 \times \cdots \times R_n) > ln(1)$	Apply natural logarithm (ln) to both sides.
3. Property Expansion	$ln(R_1) + ln(R_2) + \cdots + ln(R_n) > 0$	The log of a product equals the sum of individual logs. **(ln(1)=0)**
4. Negative Inversion	$-ln(R_1))+ \cdots +(ln(R_n))<0$	Multiply by -1 to flip the inequality. We now search for a path where the **sum** is less than zero (Negative Cycle).

Appendix C: Test Case Results & Project Links

C.1 Terminal Output

ARBITRAGE OPPORTUNITY DETECTED!

Trading Route: GBP -> EUR -> USD -> GBP

Profit Multiplier: 1.1880 (If you start with $1000, you end with $1188.00)

```python
# ==========================================
if __name__ == "__main__":
    market = ArbitrageGraph()

    # Create a loop: GBP -> EUR -> USD -> GBP
    market.add_exchange_rate('GBP', 'EUR', 1.20)  # £1 -> €1.20
    market.add_exchange_rate('EUR', 'USD', 1.10)  # €1 -> $1.10

    market.add_exchange_rate('USD', 'GBP', 0.90)  # $1 -> £0.90 (A very high rate!
    # No profit
    # Product: 1.20 * 1.10 * 0.70 = 0.924 (A loss of money)

    # Product: 1.20 * 1.10 * 0.90 = 1.188 (18.8% Profit!)

    # Start looking for arbitrage from GBP
    market.detect_arbitrage('GBP')
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\user> & C:/Users/user/anaconda3/python.exe "c:/Users/user/OneDrive - Hong Kong Metropolitan
University/桌面/Group Project/Comp task2  .py"
🚨 ARBITRAGE OPPORTUNITY DETECTED! 🚨
Trading Route: EUR -> USD -> GBP -> EUR
Profit Multiplier: 1.1880 (If you start with $1000, you end with $1188.00)
```

Image-a: code of Data Structure

Comp task2 .py

C: > Users > user > OneDrive - Hong Kong Metropolitan University > 桌

```python
class ArbitrageGraph:
    def __init__(self):
        self.graph = {}
        self.currencies = set()
        # Storing original rates just for final output display
        self.original_rates = {}

    def add_exchange_rate(self, base_currency, target_currency, rate):
        """Adds a directed, weighted edge to the graph."""
        if base_currency not in self.graph:
            self.graph[base_currency] = {}
        if target_currency not in self.graph:
            self.graph[target_currency] = {}

        self.currencies.add(base_currency)
        self.currencies.add(target_currency)

        # The mathematical transformation: convert to negative log
        weight = -math.log(rate)
        self.graph[base_currency][target_currency] = weight
        self.original_rates[(base_currency, target_currency)] = rate

    def detect_arbitrage(self, source_currency):
        """
        The Bellman-Ford Algorithm modified to detect negative weight cycles.
        """
        # Step 1: Initialize distances from the source to all other nodes as infinity
        distances = {currency: float('inf') for currency in self.currencies}
        distances[source_currency] = 0

        # We also need a predecessor map to reconstruct the arbitrage loop later
        predecessor = {currency: None for currency in self.currencies}

        # Step 2: Relax all edges |V| - 1 times
        # |V| is the total number of vertices (currencies)
        for _ in range(len(self.currencies) - 1):
            for u in self.graph:
                for v, weight in self.graph[u].items():
                    if distances[u] + weight < distances[v]:
                        distances[v] = distances[u] + weight
                        predecessor[v] = u

        # Step 3: Run one final check to detect negative weight cycles
        # If we can STILL find a shorter path, a negative cycle (arbitrage) exists
        for u in self.graph:
            for v, weight in self.graph[u].items():
                if distances[u] + weight < distances[v]:
                    print("🚨 ARBITRAGE OPPORTUNITY DETECTED! 🚨")
                    self._reconstruct_cycle(predecessor, v)
                    return True
```

Image-b: code of Algorithm

```python
def detect_arbitrage(self, source_currency):
    """
    The Bellman-Ford Algorithm modified to detect negative weight cycles.
    """
    # Step 1: Initialize distances from the source to all other nodes as infinity
    distances = {currency: float('inf') for currency in self.currencies}
    distances[source_currency] = 0

    # We also need a predecessor map to reconstruct the arbitrage loop later
    predecessor = {currency: None for currency in self.currencies}

    # Step 2: Relax all edges |V| - 1 times
    # |V| is the total number of vertices (currencies)
    for _ in range(len(self.currencies) - 1):
        for u in self.graph:
            for v, weight in self.graph[u].items():
                if distances[u] + weight < distances[v]:
                    distances[v] = distances[u] + weight
                    predecessor[v] = u

    # Step 3: Run one final check to detect negative weight cycles
    # If we can STILL find a shorter path, a negative cycle (arbitrage) exists
    for u in self.graph:
        for v, weight in self.graph[u].items():
            if distances[u] + weight < distances[v]:
                print("🚨 ARBITRAGE OPPORTUNITY DETECTED! 🚨")
                self._reconstruct_cycle(predecessor, v)
                return True

    print("No arbitrage opportunity found from this source.")
    return False
```